

ArquiControl

Documento final del proyecto

1. Portada

Nombre del proyecto: ArquiControl **Título del documento:** Documento final de análisis, diseño, implementación y validación técnica **Nombre del estudiante:** [[NOMBRE DEL ESTUDIANTE]]
Nombre del docente: [[NOMBRE DEL DOCENTE]] **Nombre de la institución:** [[NOMBRE DE LA INSTITUCIÓN]] **Programa o carrera:** [[PROGRAMA O CARRERA]] **Fecha:** [[FECHA]]

2. Tabla de contenido

1. Introducción
2. Planteamiento del problema
3. Objetivos
4. Alcance del proyecto
5. Desarrollo del proyecto por materias
 - 5.1. Aporte en Creación de Base de Datos
 - 5.2. Aporte en Gestión de Base de Datos
 - 5.3. Aporte en Ingeniería de Software
 - 5.4. Aporte en Diseño de Patrones Orientado a Objetos
6. Requerimientos del sistema
7. Casos de uso del sistema
8. Diseño de la base de datos no relacional
9. Arquitectura del sistema
10. Módulos implementados
11. Validaciones y reglas de negocio
12. Pruebas realizadas
13. Evidencia de implementación
14. Conclusiones
15. Trabajo futuro

3. Introducción

ArquiControl es un sistema web orientado a la gestión integral de proyectos para un arquitecto independiente. El proyecto surge de la necesidad de centralizar la información operativa, financiera y documental de un estudio pequeño, evitando la dispersión entre hojas de cálculo, carpetas de archivos, mensajes y registros manuales.

En el contexto de la arquitectura, la digitalización de la gestión de proyectos no solo mejora la organización de la información, sino que también fortalece la trazabilidad, la consistencia de datos y la capacidad de tomar decisiones con base en evidencia. Un sistema de este tipo resulta especialmente útil cuando un mismo proyecto involucra clientes, fases, pagos, compras, contratistas, materiales, documentos y seguimiento técnico.

La solución desarrollada implementa una arquitectura full-stack con frontend en React, backend en FastAPI y persistencia documental en MongoDB. El diseño adoptado se estructura como un monolito modular y utiliza patrones de diseño orientados a objetos para dar soporte a la lógica de dominio, especialmente dentro del agregado raíz `Proyecto`.

4. Planteamiento del problema

El arquitecto independiente suele operar con múltiples fuentes de información no centralizadas. Es común que los datos de clientes estén en una hoja de cálculo, los documentos en carpetas separadas, los avances en mensajes o fotografías aisladas y el control financiero en registros manuales. Esta situación provoca desorganización, duplicidad de datos, dificultad para medir avance real y poca trazabilidad sobre las decisiones tomadas durante el proyecto.

En este contexto, la gestión manual o dispersa de clientes, proyectos, pagos, materiales, documentos y avances afecta tanto el control administrativo como la capacidad de sustentar técnicamente el estado de cada obra o servicio. Además, cuando se requiere revisar cambios históricos o justificar la desactivación de registros, la ausencia de auditoría integrada genera un vacío importante de control.

Frente a este problema, se planteó una solución web con base de datos no relacional, adecuada para agrupar información jerárquica y operativa alrededor de un proyecto. El uso de un enfoque documental permite representar el proyecto como una unidad principal que embebe subregistros altamente relacionados, facilitando lecturas integrales y reglas de consistencia sobre una sola entidad agregada.

5. Objetivos

Objetivo general

Desarrollar un sistema web integral para un arquitecto independiente que permita gestionar clientes, proyectos y operación asociada, manteniendo control documental, financiero y trazabilidad sobre los cambios realizados.

Objetivos específicos

- Centralizar la información de clientes, proveedores, materiales y contratistas.
- Modelar el proyecto como agregado raíz en una base de datos documental.
- Gestionar fases, seguimientos, pagos, compras y documentos desde el detalle del proyecto.
- Implementar recálculo automático del avance general y del resumen financiero.
- Incorporar bitácora operativa y baja lógica con motivo obligatorio.
- Generar reportes básicos en PDF y CSV a partir del estado real del proyecto.
- Sustentar la solución mediante arquitectura modular y patrones de diseño aplicados de forma verificable.

6. Alcance del proyecto

ArquiControl cubre la autenticación del usuario administrador, la gestión de catálogos operativos, la creación y edición de proyectos, la operación embebida sobre fases, seguimientos, pagos, contratistas, compras y documentos, la bitácora global y por proyecto, el dashboard ejecutivo y la exportación de reportes básicos.

Los módulos implementados y funcionales son: autenticación, dashboard, clientes, proyectos, pagos, contratistas, proveedores, materiales, compras, documentos, bitácora y reportes. El sistema soporta procesos de negocio como seguimiento técnico, control de pagos del cliente, compras por proyecto, pagos a contratistas, trazabilidad de eventos y consulta consolidada del estado operativo de cada proyecto.

Quedan fuera del alcance actual, y deben entenderse como mejora futura, aspectos como roles y permisos multiusuario, notificaciones externas, facturación avanzada, reportes ejecutivos con branding institucional y analítica histórica más profunda.

7. Desarrollo del proyecto por materias

7.1. Aporte en Creación de Base de Datos

El proyecto inició con la necesidad de identificar las estructuras de información propias del dominio: clientes, contratistas, proveedores, materiales, proyectos, fases, pagos, seguimientos, compras y documentos. A partir de ese análisis se definió que la entidad central no sería una tabla relacional clásica, sino un documento principal llamado ``proyectos``, capaz de agrupar subdocumentos que evolucionan conjuntamente.

El paso de una visión relacional a una no relacional se justificó por la naturaleza jerárquica del dominio. Un proyecto de arquitectura necesita mantener, en una sola vista lógica, sus fases, avances, pagos, contratistas asignados, compras, documentos, alertas, indicadores y bitácora. Esta cohesión se ajusta mejor a una base de datos documental que a una estructura excesivamente fragmentada en tablas.

La base final de MongoDB quedó estructurada con colecciones independientes para ``usuarios``, ``clientes``, ``contratistas``, ``proveedores``, ``materiales`` y ``proyectos``. Dentro de ``proyectos`` se embeben ``fases``, ``seguimientos``, ``pagos``, ``contratistasAsignados``, ``compras``, ``documentos``, ``bitacora``, ``alertas`` e ``indicadores``. Las referencias externas se conservan mediante identificadores como ``clienteld``, ``contratistald``, ``proveedorId`` y ``materialId``.

``Proyectos`` se tomó como agregado raíz porque concentra el estado operativo real del negocio y permite recalcular avance, costos, saldos e indicadores sin romper la coherencia del documento. Las validaciones estructurales se implementaron con Pydantic, usando restricciones de longitud, rango numérico, obligatoriedad y validadores de reglas específicas.

7.2. Aporte en Gestión de Base de Datos

La gestión de datos se implementó mediante servicios de aplicación y repositorios por módulo. En los catálogos independientes se soportan operaciones de listar, crear y actualizar. En el módulo ``proyectos`` se implementó la gestión completa del agregado raíz y de sus subdocumentos operativos.

Los datos del sistema se administran mediante operaciones CRUD parciales o completas según el módulo. Clientes, contratistas, proveedores y materiales exponen listar, crear y actualizar. El agregado ``Proyecto`` permite crear, consultar, filtrar y actualizar, además de registrar o modificar sus subregistros asociados.

El sistema gestiona clientes, proyectos, pagos, contratistas, compras, materiales, documentos y bitácora desde una lógica consistente. Las validaciones de negocio controlan la existencia de referencias, el orden de fechas, la unicidad de ciertos atributos y la coherencia de los valores numéricos.

La baja lógica se maneja marcando ``isActive``, ``deletedAt``, ``deletedBy``, ``motivo`` y ``updatedAt``, evitando eliminar físicamente la evidencia operativa. El resumen financiero se recalcula automáticamente tras mutaciones relevantes, integrando pagos del cliente, compras y pagos a contratistas. Las consultas y filtros se realizan por texto, estado, cliente, fecha, monto y, en el caso

de la bitácora, por fecha, tipo de evento, usuario y proyecto.

7.3. Aporte en Ingeniería de Software

Desde la perspectiva de ingeniería de software, el proyecto implicó refinamiento progresivo de requerimientos, modelado del dominio, definición de casos de uso y consolidación de una arquitectura consistente. La solución final separa frontend, backend, dominio e infraestructura, y documenta explícitamente las responsabilidades de cada capa.

La arquitectura del sistema se diseñó como un monolito modular, permitiendo mantener cohesión por dominio sin introducir la complejidad operativa de microservicios. Cada módulo del backend sigue capas de `presentation`, `application`, `domain` e `infrastructure`, lo que mejora la mantenibilidad y la trazabilidad del código.

En frontend se separaron páginas, componentes, hooks y utilidades. Las vistas funcionales reales incluyen login, dashboard, módulos de catálogos, proyectos, detalle del proyecto, bitácora y feeds transversales. Esta organización favorece una lectura clara del flujo completo entre interfaz, API y persistencia.

La fase de pruebas incluyó suites backend automatizadas, validaciones visuales y regresión de flujos sensibles. El proyecto también quedó documentado con README, documentación modular y el presente documento final, lo que constituye evidencia directa del trabajo de ingeniería realizado.

7.4. Aporte en Diseño de Patrones Orientado a Objetos

El proyecto implementa patrones orientados a objetos con propósito funcional real dentro del dominio.

Repository

Definición breve: patrón que abstrae el acceso a datos mediante contratos y repositorios concretos.

Aplicación concreta: utilizado en módulos como clientes, contratistas, proveedores, materiales y proyectos para desacoplar la lógica de aplicación del acceso a MongoDB.

Archivos o módulos donde se usa: `backend/app/modules/\*/domain/repositories.py` y `backend/app/modules/\*/infrastructure/repository.py`.

Beneficio: mejora mantenibilidad, legibilidad y separación de responsabilidades.

Singleton

Definición breve: patrón que garantiza una única instancia compartida para un recurso crítico.

Aplicación concreta: `MongoConnection` centraliza la conexión a MongoDB.

Archivo donde se usa: `backend/app/core/database.py`.

Beneficio: evita múltiples clientes redundantes y simplifica el acceso a la base de datos.

Strategy

Definición breve: patrón que encapsula algoritmos intercambiables bajo una misma interfaz.

Aplicación concreta: cálculo del avance general del proyecto mediante ``AvancePorSeguimientosStrategy`` o ``AvancePorFasesStrategy``.

Archivos donde se usa: ``backend/app/modules/proyectos/domain/strategies.py`` y ``backend/app/modules/proyectos/application/service.py``.

Beneficio: permite cambiar la lógica de avance sin dispersar condicionales por el sistema.

Factory Method

Definición breve: patrón que delega la creación de objetos a fábricas especializadas según un criterio de negocio.

Aplicación concreta: construcción de pagos de tipo ``ANTICIPO``, ``PAGO_POR_FASE`` y ``PAGO_GENERAL``.

Archivos donde se usa: ``backend/app/modules/proyectos/domain/payment_factories.py`` y ``backend/app/modules/proyectos/application/service.py``.

Beneficio: centraliza reglas de construcción y reduce duplicidad de lógica en pagos.

Observer

Definición breve: patrón que permite reaccionar a eventos y ejecutar efectos derivados desacoplados del emisor principal.

Aplicación concreta: cada evento relevante del agregado ``Proyecto`` actualiza bitácora, indicadores y alertas.

Archivos donde se usa: ``backend/app/modules/proyectos/domain/observers.py`` y ``backend/app/modules/proyectos/application/service.py``.

Beneficio: conserva trazabilidad y consistencia derivada sin sobrecargar los endpoints con lógica repetida.

8. Requerimientos del sistema

Los requerimientos funcionales implementados abarcan autenticación, gestión de clientes, gestión de contratistas, gestión de proveedores, gestión de materiales, gestión de proyectos, seguimiento técnico, pagos del cliente, asignaciones y pagos a contratistas, compras, documentos, bitácora y reportes básicos.

Los requerimientos no funcionales identificables incluyen modularidad por dominio, validación de datos con Pydantic, seguridad basada en JWT, consistencia automática del resumen financiero, trazabilidad por observer, separación frontend/backend y pruebas verificables sobre URL pública.

9. Casos de uso del sistema

- autenticación del usuario mediante login;
- gestión de clientes;
- gestión de proyectos;
- seguimiento del proyecto por fase;
- gestión de pagos del cliente;

- gestión de contratistas y sus avances/pagos;
- gestión de proveedores;
- gestión de materiales;
- gestión de compras;
- gestión de documentos por URL o archivo;
- consulta de bitácora global y por proyecto;
- generación de reportes básicos.

10. Diseño de la base de datos no relacional

Las colecciones principales del sistema son `usuarios`, `clientes`, `contratistas`, `proveedores`, `materiales` y `proyectos`. El documento `proyectos` concentra la operación del negocio y embebe fases, seguimientos, pagos, contratistas asignados, compras, documentos, resumen financiero, bitácora, alertas e indicadores.

Los subdocumentos embebidos permiten mantener una visión integral del proyecto. Las referencias externas se resuelven por identificadores hacia colecciones maestras. Entre las reglas de negocio importantes están: validación de cliente existente al crear proyecto, pagos por fase con fase obligatoria, compras con detalle obligatorio, fase no eliminable si tiene seguimientos o pagos activos y cierre de proyecto bloqueado con saldo pendiente importante.

MongoDB fue adecuada para este sistema porque el dominio gira alrededor de un agregado documental complejo y altamente cohesionado. Esta estructura permite agrupar información operativa sin forzar un diseño excesivamente distribuido en tablas relacionales.

11. Arquitectura del sistema

El sistema usa un estilo arquitectónico de monolito modular. El backend organiza sus módulos por dominio y capas. El frontend se estructura por páginas, componentes, hooks y utilidades. MongoDB actúa como persistencia principal y el sistema de archivos soporta cargas documentales en `UPLOADS\_DIR`.

La relación entre frontend, backend, MongoDB y archivos es directa: el frontend consume la API FastAPI, el backend aplica reglas de dominio y accede a MongoDB mediante repositorios, y los archivos cargados se guardan en disco y se sirven por una ruta autenticada.

La estructura real de carpetas incluye `backend/app/core`, `backend/app/modules`, `backend/app/seed`, `backend/app/shared`, `frontend/src/pages`, `frontend/src/components`, `docs`, `memory` y `uploads`.

12. Módulos implementados

Autenticación Propósito: controlar acceso al sistema. Funcionalidades: login, sesión actual. Relación con el negocio: protege el acceso al panel técnico.

Dashboard Propósito: consolidar métricas y estado general. Funcionalidades: alertas, saldos, avances, pagos y compras recientes, bitácora reciente. Relación con el negocio: permite visión ejecutiva del portafolio.

Clientes Propósito: mantener base de clientes. Funcionalidades: listar, crear y actualizar. Relación con el negocio: vincula proyectos a clientes reales.

Proyectos Propósito: gestionar el agregado raíz. Funcionalidades: crear, editar, filtrar y consultar detalle. Relación con el negocio: concentra la operación del estudio.

Seguimiento Propósito: registrar avances técnicos por fase. Funcionalidades: crear, editar y baja lógica. Relación con el negocio: documenta progreso del proyecto.

Pagos Propósito: controlar pagos del cliente y su efecto financiero. Funcionalidades: registrar, editar, baja lógica y feed global. Relación con el negocio: seguimiento de ingresos y saldo pendiente.

Contratistas Propósito: asignar y controlar especialistas externos. Funcionalidades: catálogo, asignaciones, avances, pagos y baja lógica condicionada. Relación con el negocio: gestión de ejecución técnica externa.

Proveedores Propósito: mantener red de suministro. Funcionalidades: listar, crear y actualizar. Relación con el negocio: soporte de compras por proyecto.

Materiales Propósito: mantener referencias de compra. Funcionalidades: listar, crear y actualizar. Relación con el negocio: detalle técnico y financiero de compras.

Compras Propósito: registrar egresos por proyecto. Funcionalidades: crear, editar, baja lógica y feed global. Relación con el negocio: control de costos.

Documentos Propósito: gestionar evidencia y archivos. Funcionalidades: URL, upload, edición, apertura y baja lógica. Relación con el negocio: soporte documental del proyecto.

Bitácora Propósito: registrar trazabilidad operativa. Funcionalidades: vista global, vista por proyecto, filtros y detalle expandible. Relación con el negocio: auditoría y control.

Reportes Propósito: generar salidas entregables. Funcionalidades: PDF resumen para cliente, CSV de pagos/compras, CSV de pagos a contratistas. Relación con el negocio: comunicación y control documental.

13. Validaciones y reglas de negocio

El sistema valida datos básicos, correos, mínimos de longitud, números no negativos y porcentajes válidos. También valida orden de fechas en proyectos, fases y asignaciones. Los pagos por fase exigen `faseId`; los seguimientos deben asociarse a una fase existente; las compras requieren al menos un detalle; y el proyecto no puede cerrarse si conserva saldo pendiente significativo.

La baja lógica exige motivo obligatorio y deja trazabilidad en bitácora. Cada evento relevante registra información de usuario, fecha, subregistro, acción y motivo cuando aplica. El resumen financiero se recalcula automáticamente a partir de pagos, compras y pagos a contratistas.

14. Pruebas realizadas

Se ejecutaron pruebas backend automatizadas en `test\_arquicontrol\_api.py` y `test\_observer\_traceability.py`. La evidencia más reciente documenta 24 pruebas backend aprobadas. También se realizaron smoke tests y validaciones automatizadas de interfaz sobre

login, dashboard, bitácora, detalle del proyecto, modal de baja lógica y descargas de reportes.

Durante el desarrollo se corrigieron errores importantes, especialmente uno relacionado con la baja lógica: inicialmente un prompt cancelado podía seguir disparando la eliminación. El flujo fue corregido mediante un modal formal y una validación backend explícita del motivo obligatorio.

15. Evidencia de implementación

Las pantallas reales del sistema son: login, dashboard, clientes, contratistas, proveedores, materiales, proyectos, detalle del proyecto, pagos, compras, documentos y bitácora.

El flujo funcional demostrable de punta a punta incluye:

- iniciar sesión;
- consultar dashboard;
- crear o editar catálogos;
- crear o editar proyectos;
- registrar seguimientos, pagos, contratistas, compras y documentos;
- revisar la bitácora global y por proyecto;
- ejecutar bajas lógicas con motivo;
- descargar reportes PDF y CSV.

Las operaciones descritas están respaldadas por endpoints reales, pruebas backend y validaciones funcionales registradas en ``test_reports/``.

16. Conclusiones

ArquiControl logró consolidar una solución funcional para la gestión de proyectos de un arquitecto independiente, integrando operación, trazabilidad, control financiero y soporte documental en una sola plataforma. Desde el punto de vista académico, el proyecto evidencia la aplicación real de conceptos de bases de datos, gestión de datos, ingeniería de software y patrones orientados a objetos.

El sistema aporta valor técnico al implementar una arquitectura modular, una base de datos no relacional coherente con el dominio y reglas de negocio que mantienen consistencia sobre el agregado principal. Con ello, el objetivo general del proyecto puede considerarse cumplido dentro del alcance definido.

17. Trabajo futuro

Como trabajo futuro, y no como funcionalidad actual, pueden considerarse:

- roles y permisos multiusuario;
- reportes con formato institucional avanzado;
- buscador global más amplio;
- indicadores históricos más profundos;
- mejoras de versionado documental;

- integraciones externas complementarias.

Estas mejoras no forman parte del estado actual implementado y se mencionan únicamente como proyección evolutiva.